**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM**
**KHOA KỸ THUẬT MÁY TÍNH**

CE119 – Thực hành Kiến trúc Máy tính

# LAB 02
## MẢNG, CHUỖI, THỦ TỤC VÀ NGĂN XẾP

Thời lượng: 180 phút · Công cụ: MARS 4.5

| | | | |
|---|---|---|---|
| **Họ và tên** | Huỳnh Phạm Ngọc Thắng | **MSSV** | 25521672 |
| **Lớp** | CE119.R11.VMTN.1 | **Ngày thực hành** | 20/10/2026 |
| **Tự chấm (/10)** | 9 | **Liên kết GitHub** | https://github.com/hpnt1912/COMPUTER-ARCHITECTURE/tree/main/mars_mips/lab2 |

## 1. TỔNG QUAN BUỔI THỰC HÀNH

### Mục đích

Buổi thực hành xây dựng một bộ tiện ích nhỏ cho thiết bị nhúng: thống kê mảng điểm số và kiểm tra cặp ngoặc trong chuỗi bằng ngăn xếp (stack). Sinh viên làm việc trực tiếp với bộ nhớ (địa chỉ, offset, căn lề, thứ tự byte); được hướng dẫn cách viết thủ tục, tức là đơn vị mã có tên, nhận tham số và trả kết quả theo quy ước thanh ghi; và dùng ngăn xếp như một cấu trúc dữ liệu "vào sau, ra trước", quan sát hoạt động của nó trên MARS thay vì chỉ học thuộc quy tắc.

### Sau buổi thực hành, sinh viên có thể

- tính đúng địa chỉ và offset khi truy cập mảng word, mảng byte và chuỗi bằng `lw`, `sw`, `lb`, `lbu`, `sb`; giải thích kết quả quan sát được trong Data Segment (căn lề, thứ tự byte, mở rộng dấu);
- giải thích cơ chế gọi và trả về của thủ tục (`jal`, `jr $ra`, thanh ghi `$ra`); viết thủ tục lá nhận mảng làm tham số và trả về một hoặc hai giá trị theo quy ước `$a`, `$v`, `$t`;
- thực hiện thao tác push và pop bằng `addi`, `sw`, `lw` trên `$sp`; giải thích vì sao ngăn xếp phù hợp với bài toán kiểm tra ngoặc;
- giữ `$sp` cân bằng trên mọi đường thoát của một thủ tục;
- trực quan hóa dữ liệu thống kê bằng ký tự văn bản (phần mở rộng).

**Bảng 1.** *Phân bổ nội dung, thời gian và điểm*

| Phần | Nội dung | Thời gian gợi ý | Điểm |
|---|---|---|---|
| Phần 0 | Chuẩn bị môi trường, tham số cá nhân | 10 phút | – |
| TN1 | Bản đồ bộ nhớ: địa chỉ, căn lề, thứ tự byte, `lb`/`lbu` | 20 phút | 1,5 |
| TN2 | Thủ tục và mảng: hướng dẫn thủ tục, thống kê điểm bằng thủ tục lá | 55 phút | 3,0 |
| TN3 | Ngăn xếp: kiểm tra ngoặc cân bằng | 60 phút | 3,5 |
| Kết thúc | Nộp bài, phản tư, vấn đáp | 20 phút | 2,0 |
| Mở rộng | Trực quan hóa phổ điểm bằng ký tự `\|` (dành cho sinh viên hoàn thành sớm) | 15 phút | +1,0 |
| **Tổng** | | **180 phút** | **10,0** |

**Quy ước làm bài**

- **Ô nền vàng nhạt** là chỗ điền câu trả lời (chữ nhập vào ô có màu đỏ); **khung nét đứt** là nơi dán ảnh chụp màn hình MARS (ảnh phải thấy rõ thanh ghi hoặc vùng nhớ liên quan).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Điền dự đoán **trước** khi chạy. Dự đoán sai không bị trừ điểm; phần giải thích mới được chấm.
- Câu hỏi được gắn mức nhận thức: Hiểu, Áp dụng, Phân tích, Đánh giá.
- Chương trình nộp là **một file .asm duy nhất**. Mỗi thủ tục có chú thích đầu: tham số, giá trị trả về, thanh ghi sử dụng. Chuỗi in ra và chú thích dùng ASCII không dấu.
- Mọi giá trị địa chỉ và thanh ghi ghi ở dạng thập lục phân (hex) 8 chữ số, trừ khi đề yêu cầu khác.

## PHẦN 0 – CHUẨN BỊ MÔI TRƯỜNG (10 phút)

**Bảng 2.** *Kỹ năng cần thực hiện được trước khi làm thí nghiệm*

| ✓ | Kỹ năng | Gợi ý |
|---|---|---|
| ✓ | Bật khởi tạo PC tại nhãn `main` | Menu Settings → "Initialize Program Counter to global 'main' if defined" |
| ✓ | Hiển thị Data Segment dạng hex và ASCII | Các ô chọn ở cuối cửa sổ Data Segment: Hexadecimal Values, ASCII |
| ✓ | Chuyển vùng xem bộ nhớ sang `.data` và sang "current $sp" | Danh sách thả xuống ở cuối cửa sổ Data Segment |
| ✓ | Xem địa chỉ các nhãn | Ô Labels ở cửa sổ Execute |
| ✓ | Chạy từng bước, lùi bước, đặt breakpoint | Các nút Step, Backstep; ô Bkpt ở cột đầu của Text Segment |

### Bước 0.1 – Kiểm tra nhanh

Nhập và chạy chương trình sau để làm quen với syscall in số thập lục phân (mã 34) và in ký tự (mã 11). Ghi lại hai giá trị in ra; giá trị `$sp` được dùng lại ở TN3.

```
.data
w1:   .word 0
.text
main:
    move $a0, $sp
    li   $v0, 34
    syscall
    li   $a0, 10
    li   $v0, 11
    syscall
    la   $a0, w1
    li   $v0, 34
    syscall
    li   $v0, 10
    syscall
```

**Bảng 3.** *Giá trị đo ở Bước 0.1*

| Đại lượng | Giá trị (hex) |
|---|---|
| `$sp` khi chương trình bắt đầu | 0x7fffeffc |
| Địa chỉ nhãn `w1` | 0x10010000 |

### Bước 0.2 – Tham số cá nhân

Gọi **d** là chữ số cuối cùng của MSSV. Điền các tham số cá nhân dùng trong các thí nghiệm. Cơ sở của mảng điểm là dãy **A0 = {6, 3, 8, 9, 2, 5, 7, 2, 6, 9, 4, 3}** (12 phần tử); mảng điểm của bạn là S[i] = (A0[i] + d) mod 11, mọi phần tử đều thuộc [0, 10].

**Bảng 4.** *Tham số cá nhân*

| Tham số | Công thức | Giá trị của bạn |
|---|---|---|
| d | chữ số cuối của MSSV | 2 |
| Mảng điểm S[0..11] | S[i] = (A0[i] + d) mod 11 | {8, 5, 10, 0, 4, 7, 9, 4, 8, 0, 6, 5} |
| k | d + 3 | 5 |

**Quy ước đặt tên file nộp:** `CE119_Lab02_<MSSV>.asm` cho mã nguồn và `CE119_Lab02_<MSSV>.docx` cho báo cáo này (đã điền).

## TN1 – BẢN ĐỒ BỘ NHỚ (20 phút · 1,5 điểm)

### Tình huống

Một bản ghi cấu hình của thiết bị nhúng gồm một word, ba byte điều khiển và một chuỗi định danh. Trước khi viết thủ tục duyệt mảng và chuỗi, cần xác định chính xác vị trí, giá trị và thứ tự của từng byte trong bộ nhớ.

### Nhắc lại hai lệnh nạp byte

- `lb` nạp một byte rồi **mở rộng dấu** (sign-extend): bit 7 của byte được chép vào 24 bit cao.
- `lbu` nạp một byte rồi **mở rộng không dấu** (zero-extend): 24 bit cao bằng 0.

**Bước 1.1.** Nhập chương trình sau, assemble, và mở Data Segment (bật hex và ASCII).

```
.data
w1:  .word 0x8A7B6C5D
b1:  .byte 0x80, 0x7F, 0xFF
     .align 2
w2:  .word 1
s1:  .asciiz "IT012"
.text
main:
    la   $s0, w1
    lw   $t0, 0($s0)       # (a)
    lb   $t0, 0($s0)       # (b)
    lb   $t0, 3($s0)       # (c)
    lbu  $t0, 3($s0)       # (d)
    lb   $t0, 4($s0)       # (e)
    lbu  $t0, 4($s0)       # (f)
    lw   $t0, 4($s0)       # (g)
    li   $t1, 0xAB
    sb   $t1, 1($s0)       # (h)
    lw   $t0, 0($s0)       # (i)
    li   $v0, 10
    syscall
```

**Bước 1.2 (DỰ ĐOÁN → QUAN SÁT).** Điền Bảng 5 với địa chỉ dự đoán của từng nhãn (vùng `.data` bắt đầu tại `0x10010000`), sau đó đối chiếu với cửa sổ Labels.

**Bảng 5.** *Địa chỉ các nhãn*

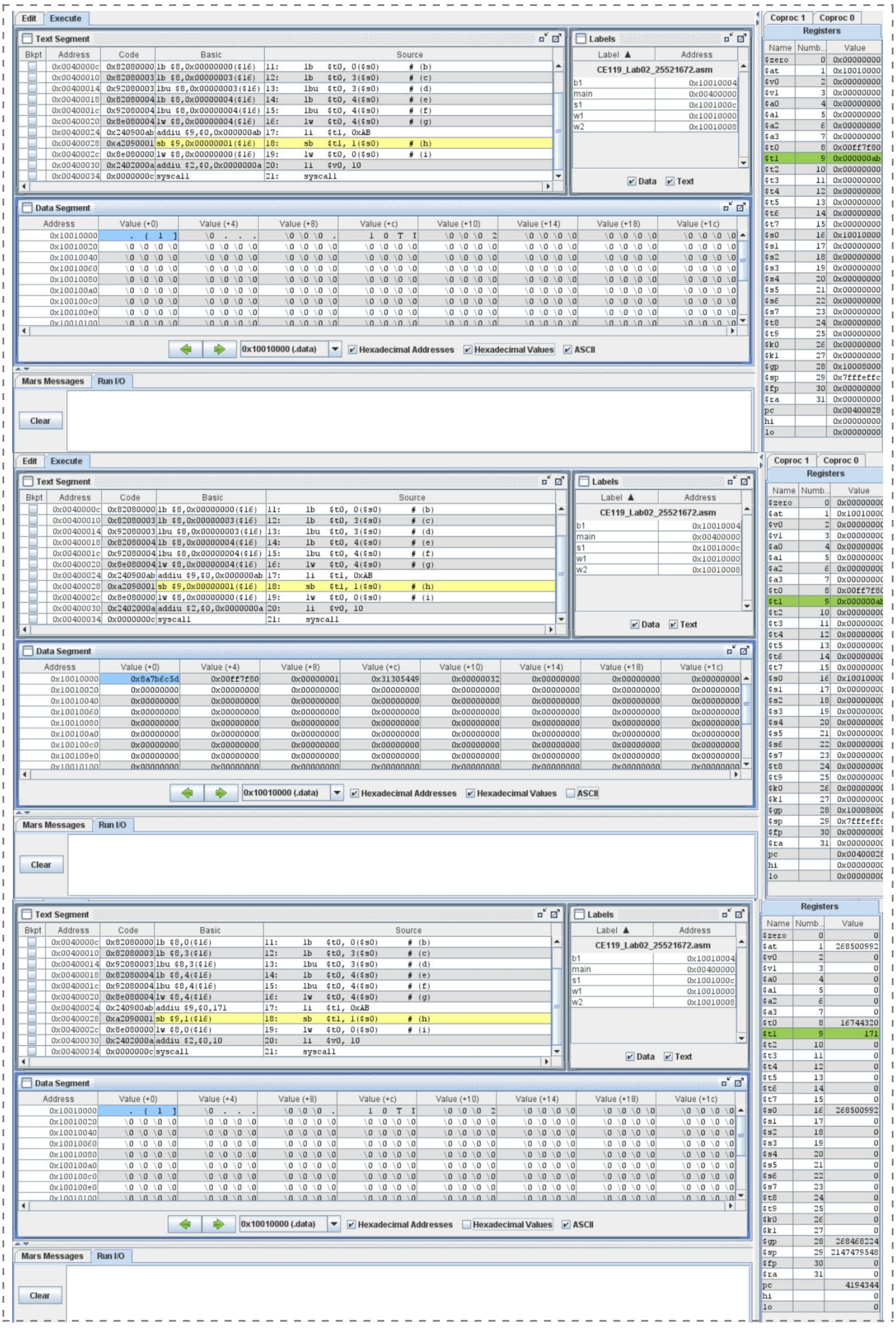
| Nhãn | Dự đoán | Quan sát |
|---|---|---|
| `w1` | 0x10010000 | 0x10010000 |
| `b1` | 0x10010004 | 0x10010004 |
| `w2` | 0x10010008 | 0x10010008 |
| `s1` | 0x1001000C | 0x1001000C |

**Bước 1.3 (DỰ ĐOÁN → QUAN SÁT).** Với mỗi dòng (a)–(i), ghi giá trị `$t0` mà bạn dự đoán theo mô hình bộ nhớ được trình bày trong slide Chương 2, rồi chạy từng bước (Step) và ghi giá trị quan sát. Dòng (i) đọc lại word tại `w1` sau khi (h) đã ghi.

**Bảng 6.** *Giá trị* `$t0` *sau mỗi lệnh*

| Dòng | Lệnh | Dự đoán `$t0` | Quan sát `$t0` | Khớp (✓/✗) |
|---|---|---|---|---|
| (a) | `lw $t0, 0($s0)` | 0x8A7B6C5D | 0x8A7B6C5D | ✓ |
| (b) | `lb $t0, 0($s0)` | 0x0000005D | 0x0000005D | ✓ |
| (c) | `lb $t0, 3($s0)` | 0xFFFFFF8A | 0xFFFFFF8A | ✓ |
| (d) | `lbu $t0, 3($s0)` | 0x0000008A | 0x0000008A | ✓ |
| (e) | `lb $t0, 4($s0)` | 0xFFFFFF80 | 0xFFFFFF80 | ✓ |
| (f) | `lbu $t0, 4($s0)` | 0x00000080 | 0x00000080 | ✓ |
| (g) | `lw $t0, 4($s0)` | 0x00FF7F80 | 0x00FF7F80 | ✓ |
| (h)+(i) | `sb $t1, 1($s0)` rồi `lw $t0, 0($s0)` | 0x000000AB | 0x8A7BAB5D | ✗ |

Dán ảnh Data Segment (hex và ASCII) vùng từ `w1` đến hết `s1`, trước khi chạy dòng (h):


Edit
Execute
Text Segment
Bkpt Address Code Basic Source
0x0040000c 0x82080000 lb $8,0x00000000($16) 11: lb $t0, 0($s0) # (b)
0x00400010 0x82080003 lb $8,0x00000003($16) 12: lb $t0, 3($s0) # (c)
0x00400014 0x92080003 lbu $8,0x00000003($16) 13: lbu $t0, 3($s0) # (d)
0x00400018 0x82080004 lb $8,0x00000004($16) 14: lb $t0, 4($s0) # (e)
0x0040001c 0x92080004 lbu $8,0x00000004($16) 15: lbu $t0, 4($s0) # (f)
0x00400020 0x8e080004 lw $8,0x00000004($16) 16: lw $t0, 4($s0) # (g)
0x00400024 0x240900ab addiu $9,$0,0x000000ab 17: li $t1, 0xAB
0x00400028 0xa2090001 sb $9,0x00000001($16) 18: sb $t1, 1($s0) # (h)
0x0040002c 0x8e080000 lw $8,0x00000000($16) 19: lw $t0, 0($s0) # (i)
0x00400030 0x2402000a addiu $2,$0,0x0000000a 20: li $v0, 10
0x00400034 0x0000000c syscall 21: syscall
Labels
Label Address
CE119_Lab02_25521672.asm
b1 0x10010004
main 0x00400000
s1 0x1001000c
w1 0x10010000
w2 0x10010008
Data Text
Data Segment
Address Value (+0) Value (+4) Value (+8) Value (+c) Value (+10) Value (+14) Value (+18) Value (+1c)
0x10010000 0x10010020 0x10010040 0x10010060 0x10010080 0x100100a0 0x100100c0 0x100100e0 0x10010100
0x10010000 (.data)
Hexadecimal Addresses
Hexadecimal Values
ASCII
Mars Messages
Run I/O
Clear
Coproc 1
Coproc 0
Registers
Name Numb... Value
$zero 0 0x00000000
$at 1 0x10010000
$v0 2 0x00000000
$v1 3 0x00000000
$a0 4 0x00000000
$a1 5 0x00000000
$a2 6 0x00000000
$a3 7 0x00000000
$t0 8 0x00ff7f80
$t1 9 0x000000ab
$t2 10 0x00000000
$t3 11 0x00000000
$t4 12 0x00000000
$t5 13 0x00000000
$t6 14 0x00000000
$t7 15 0x00000000
$s0 16 0x10010000
$s1 17 0x00000000
$s2 18 0x00000000
$s3 19 0x00000000
$s4 20 0x00000000
$s5 21 0x00000000
$s6 22 0x00000000
$s7 23 0x00000000
$t8 24 0x00000000
$t9 25 0x00000000
$k0 26 0x00000000
$k1 27 0x00000000
$gp 28 0x10008000
$sp 29 0x7fffeffc
$fp 30 0x00000000
$ra 31 0x00000000
pc 0x00400028
hi 0x00000000
lo 0x00000000
0x8a7b6c5d 0x00ff7f80 0x00000001 0x31305449 0x00000032
0x0040000c 0x82080000 lb $8,0($16) 11: lb $t0, 0($s0) # (b)
0x00400010 0x82080003 lb $8,3($16) 12: lb $t0, 3($s0) # (c)
0x00400014 0x92080003 lbu $8,3($16) 13: lbu $t0, 3($s0) # (d)
0x00400018 0x82080004 lb $8,4($16) 14: lb $t0, 4($s0) # (e)
0x0040001c 0x92080004 lbu $8,4($16) 15: lbu $t0, 4($s0) # (f)
0x00400020 0x8e080004 lw $8,4($16) 16: lw $t0, 4($s0) # (g)
0x00400024 0x240900ab addiu $9,$0,171 17: li $t1, 0xAB
0x00400028 0xa2090001 sb $9,1($16) 18: sb $t1, 1($s0) # (h)
0x0040002c 0x8e080000 lw $8,0($16) 19: lw $t0, 0($s0) # (i)
0x00400030 0x2402000a addiu $2,$0,10 20: li $v0, 10
0x00400034 0x0000000c syscall 21: syscall
0x10010000 (.data)
Hexadecimal Addresses
Hexadecimal Values
ASCII
Mars Messages
Run I/O
Clear
Registers
Name Numb... Value
$zero 0 0
$at 1 268500992
$v0 2 0
$v1 3 0
$a0 4 0
$a1 5 0
$a2 6 0
$a3 7 0
$t0 8 16744320
$t1 9 171
$t2 10 0
$t3 11 0
$t4 12 0
$t5 13 0
$t6 14 0
$t7 15 0
$s0 16 268500992
$s1 17 0
$s2 18 0
$s3 19 0
$s4 20 0
$s5 21 0
$s6 22 0
$s7 23 0
$t8 24 0
$t9 25 0
$k0 26 0
$k1 27 0
$gp 28 268468224
$sp 29 2147479548
$fp 30 0
$ra 31 0
pc 4194344
hi 0
lo 0

**Câu 1.1 (Phân tích).** Giải thích vì sao địa chỉ của `w2` không nằm ngay sau byte cuối của `b1`. Byte nằm ở khoảng trống đó có giá trị bao nhiêu và do đâu mà có?

Vì b1 gồm 3 byte, chiếm các địa chỉ 0x10010004-0x10010006, nên byte kế tiếp là 0x10010007 vì nó ko phải bội của 4 nên có dòng align 2 là chuyển địa chỉ byte sang bội 2^2 là 4 nên nó chuyển sang 0x10010008 vì vậy nên w2 mới truy cập được bằng lw/sw vì 1 word phải là bội của 4. byte ở khoảng trống đó có giá trị là 0x00 vì nó là byte đệm do căn lề sinh ra, có giá trị 0 vì vùng nhớ chưa được gán dữ liệu nên MARS khởi tạo bằng 0

**Câu 1.2 (Phân tích).** So sánh kết quả của (e) và (f). Dựa trên phần số có dấu và không dấu trong slide, giải thích cơ chế tạo ra từng kết quả.

e: 0xFFFFFF80, f: 0x00000080

Vì $s0 chứa địa chỉ của w1 là 0x10010000; word tại địa chỉ đó là 0x8A7B6C5D.

--> cả 2 lệnh đều đọc 1 byte tại địa chỉ 10010000 + 4 = 0x10010004 là byte đầu tiền của b1 là 80(ở trong 0x00FF7F80) --> bit 7 là số 1 (10000000)

--> câu e là lệnh lb --> thêm số 1(vì bit 7 = 1) vào 24 bit cao --> 0xFFFFFF80

--> câu f là lệnh lbu --> mặc định thêm số 0 vào 24 bit cao --> 0x00000080

Vì theo lb thì 0x80 = -128(ở dạng bù 2) khi mở rộng thành 24 bit 1 nó vẫn là -128, còn lbu thì 0x80 là 128 khi mở rộng 24 bit 0 thì vẫn 128

**Câu 1.3 (Phân tích).** Đối chiếu cột dự đoán với cột quan sát ở (b), (c), (g) và (i). Nếu có sai khác, hãy chỉ ra chính xác sai khác nằm ở đâu và giải thích nguyên nhân bằng dữ liệu bạn thấy trong Data Segment.

Sai ở (i) vì dự đoán là 0x000000AB nhưng quan sát là 0x8A7BAB5D vì sau lệnh h thì $t1 đang là 0x000000AB, địa chỉ là 0x1001000 + 1 = 0x1001001, byte của $s0 tại đó là 6C --> thay 6C bằng AB

Nên sau lệnh i, $t0 = 0x8A7BAB5D

**Bước 1.4 (DỰ ĐOÁN → QUAN SÁT).** Thêm dòng `lw $t0, 1($s0)` ngay trước `li $v0, 10`, assemble và chạy. Ghi lại hiện tượng.

**Câu 1.4 (Áp dụng).** Chép nguyên văn thông báo của MARS và nêu điều kiện của địa chỉ bị vi phạm. Với một mảng word có địa chỉ nền `$s3`, công thức địa chỉ phần tử `i` đảm bảo điều kiện này là gì?

Error in D:\github_git\repo\COMPUTER-ARCHITECTURE\mars_mips\lab2\CE119_Lab02_25521672.asm line 20: Runtime exception at 0x00400030: fetch address not aligned on word boundary 0x10010001

Go: execution terminated with errors.

Vi phạm vì để dùng lệnh lw/sw để truy cập địa chỉ thì địa chỉ đó phải là bội số của 4. 0x10010001 không chia hết cho 4 nên lỗi.

Nếu có địa chỉ nền $s3 thì để truy cập địa chỉ phần tử I là:

Địa chỉ A[i] = $s3 + 4 * i

## TN2 – THỦ TỤC VÀ MẢNG: THỐNG KÊ ĐIỂM (55 phút · 3,0 điểm)

### Tình huống

Hệ thống quản lý lớp học cần thống kê mảng điểm nguyên (thang 0–10) trước khi xuất báo cáo: điểm thấp nhất, điểm cao nhất và số sinh viên trong từng nhóm xếp loại. Phần mềm được chia thành các thủ tục nhỏ, mỗi thủ tục làm một việc và có thể kiểm thử riêng. Trong thí nghiệm này, trước hết bạn làm quen với thủ tục qua hai ví dụ mẫu chạy được, sau đó tự viết hai thủ tục thống kê.

### A. Kiến thức nền: thủ tục

**Thủ tục (procedure)** là một đoạn mã có tên (nhãn), được gọi từ nhiều nơi trong chương trình và khi chạy xong thì quay về đúng chỗ đã gọi. Dùng thủ tục giúp chia chương trình lớn thành các phần nhỏ, kiểm thử riêng từng phần và tái sử dụng.

Việc gọi và quay về do hai lệnh đảm nhiệm:

- `jal nhãn` (jump and link): ghi địa chỉ của lệnh nằm **ngay sau** `jal` vào thanh ghi `$ra` (return address), rồi nhảy đến `nhãn`.
- `jr $ra` (jump register): nhảy đến địa chỉ đang chứa trong `$ra`, tức là quay về lệnh nằm ngay sau `jal`.

Người gọi và thủ tục trao đổi dữ liệu qua thanh ghi theo một quy ước chung:

#### Quy ước thanh ghi khi gọi thủ tục

- `$a0`–`$a3`: tham số truyền vào (tối đa bốn). Thủ tục được phép thay đổi các thanh ghi này.
- `$v0`, `$v1`: giá trị trả về. (`$v0` cũng chứa mã syscall, `$a0` cũng chứa tham số của syscall: lệnh `syscall` làm thay đổi chúng.)
- `$t0`–`$t9`: thanh ghi tạm. Thủ tục dùng tự do; người gọi **không được** trông cậy vào giá trị của chúng sau lời gọi.
- `$s0`–`$s7`: thanh ghi "giữ nguyên qua lời gọi". Thủ tục nào dùng đến chúng phải trả lại nguyên giá trị khi return. Các thủ tục trong bài này không dùng `$s`, nên chương trình chính có thể cất giá trị vào `$s0`–`$s7` để dùng sau các lời gọi.
- `$ra`: địa chỉ trả về, do `jal` ghi.

Thủ tục chỉ dùng `$a`, `$v`, `$t` và **không gọi thủ tục khác** (không có `jal` bên trong) gọi là **thủ tục lá (leaf procedure)**. Với thủ tục lá, `$ra` không bị thay đổi nên chỉ cần `jr $ra` ở cuối. Trường hợp thủ tục gọi thủ tục khác cần thêm kỹ thuật lưu `$ra`, được học ở bài thực hành sau; trong bài này mọi thủ tục đều là thủ tục lá.

Quy tắc viết: mỗi thủ tục bắt đầu bằng một nhãn riêng và kết thúc bằng `jr $ra`; nhãn bên trong thủ tục (vòng lặp, nhánh) nên có tiền tố là tên thủ tục (ví dụ `max2_end`) vì nhãn phải duy nhất trong cả file; chương trình chính phải kết thúc bằng syscall thoát (mã 10) **trước** đoạn mã thủ tục, nếu không chương trình sẽ "chạy lọt" vào thủ tục.

### B. Ví dụ mẫu 1: thủ tục có hai tham số và một kết quả

Thủ tục `max2` nhận hai số nguyên có dấu trong `$a0`, `$a1` và trả về số lớn hơn trong `$v0`. Chương trình chính nạp tham số, gọi thủ tục bằng `jal`, rồi in kết quả.

```
.text
main:
    li   $a0, 17
    li   $a1, 25
    jal  max2
    move $a0, $v0
    li   $v0, 1
    syscall
    li   $v0, 10
    syscall

# max2: tra ve so lon hon trong hai so nguyen co dau
# Tham so : $a0 = a, $a1 = b
# Tra ve  : $v0 = max(a, b)
max2:
    move $v0, $a0
    slt  $t0, $a0, $a1
    beq  $t0, $zero, max2_end
    move $v0, $a1
max2_end:
    jr   $ra
```

**Bước 2.1 (DỰ ĐOÁN → QUAN SÁT).** Nhập chương trình, assemble và xem cột Address trong Text Segment. Dựa vào địa chỉ các lệnh, điền cột "Dự đoán" của Bảng 7; sau đó chạy từng bước (Step) và điền cột "Quan sát".

**Bảng 7.** *Gọi và trả về của* `max2`

| Thời điểm | Dự đoán | Quan sát |
|---|---|---|
| PC ngay sau khi `jal max2` thực thi | 0x00400020 | 0x00400020 |
| `$ra` ngay sau khi `jal max2` thực thi | 0x0040000C | 0x0040000C |
| `$v0` ngay trước khi `jr $ra` thực thi | 0x0040002C | 0x00000019 |
| PC ngay sau khi `jr $ra` thực thi | 0x0040000C | 0x0040000C |

**Câu 2.1 (Hiểu).** Dựa vào Bảng 7, mô tả bằng lời cách `jal` và `jr $ra` phối hợp để chương trình quay về đúng chỗ. Nếu chương trình chính gọi `max2` ở hai vị trí khác nhau thì giá trị `$ra` ở hai lần gọi có giống nhau không? Điều đó giúp một thủ tục được gọi từ nhiều nơi như thế nào?

- Cách phối hợp: đầu tiên khi jal max2 thì nó sẽ nhảy tới lệnh trong max2 còn $ra sẽ nắm giữ địa chỉ sau lệnh jal max2 (move $a0, $v0). khi chạy max2, nếu $a0 < $a1 = 0 thì nó sẽ quay về lệnh mà $ra chứa và xuất ra $a0 (vì đã ghi $v0 = $a0 từ trước) vì $a0 là max còn lại nếu $a0 < $a1 = 1 thì nó sẽ xuất luôn ra sẽ ghi $v0 là giá trị $a1 và xuất ra $v0 luôn
- giá trị $ra ở hai lần gọi giống nhau. Nó giữ địa chỉ sau lệnh jal.
- nó giúp 1 thủ tục gọi nhiều nơi bằng cách chỉ cần jr $ra thì lập từng sẽ trỏ tới $ra (là thanh ghi nắm giữa địa chỉ ngay sau lệnh jal)

## C. Ví dụ mẫu 2: truyền mảng cho thủ tục

Một thủ tục nhận mảng thông qua **địa chỉ đầu mảng** và **số phần tử**. Thủ tục `sum_array` dưới đây duyệt mảng bằng con trỏ: sau mỗi phần tử kiểu word, con trỏ `$a0` tăng thêm 4. (Cách khác là giữ chỉ số `i` và tính địa chỉ bằng `sll` và `add` như trong slide.)

```
.data
arr: .word 4, 9, 1, 6
.text
main:
    la   $a0, arr
    li   $a1, 4
    jal  sum_array
    move $a0, $v0
    li   $v0, 1
    syscall
    li   $v0, 10
    syscall

# sum_array: tong cac phan tu cua mang word
# Tham so : $a0 = dia chi mang, $a1 = n (n >= 0)
# Tra ve  : $v0 = tong
sum_array:
    move $v0, $zero
sa_loop:
    beq  $a1, $zero, sa_done
    lw   $t0, 0($a0)
    add  $v0, $v0, $t0
    addi $a0, $a0, 4
    addi $a1, $a1, -1
    j    sa_loop
sa_done:
    jr   $ra
```

**Bước 2.2 (DỰ ĐOÁN → QUAN SÁT).** Dự đoán các đại lượng trong Bảng 8. Chạy chương trình, đặt breakpoint tại lệnh `move $a0, $v0` (ngay sau lời gọi) để đọc `$a0` và `$a1` khi thủ tục vừa trả về, rồi chạy tiếp để xem số in ra.

**Bảng 8.** *Kết quả của* `sum_array`

| Đại lượng | Dự đoán | Quan sát |
|---|---|---|
| Số in ra | 20 | 20 |
| `$a0` khi thủ tục vừa trả về (hex) | 0x0000000C | 0x10010010 |
| `$a1` khi thủ tục vừa trả về (hex) | 12 | 0x00000000 |

**Câu 2.2 (Phân tích).** So sánh `$a0` và `$a1` khi thủ tục vừa trả về với giá trị lúc gọi. Quy ước thanh ghi cho phép điều này hay không? Hệ quả đối với chương trình chính là gì nếu nó cần dùng lại địa chỉ mảng sau lời gọi, và có những cách nào để xử lý?

Giá trị lúc gọi: $a0 = 0x10010000 , $a1 = 0x00000004

Giá trị trả về: $a0 = 0x10010010, $a1 = 0x00000000

- quy ước thanh ghi cho phép vì thay đổi giá trị ở $a0 với $a1 là bình thường

- hệ quả: $a0 và $a1 thay đổi dẫn tới không thể tái sử dụng vì $a0 không còn chứa địa chỉ arr và $a1 không còn là độ dài mảng.

- xử lý: lưu trước địa chỉ $a0 trước khi dùng, nạp lại địa chỉ $a0 sau khi dùng, sao chép sang thanh ghi tạm để dùng

## D. Nhiệm vụ: thống kê điểm

**Nhóm xếp loại:** nhóm 0 gồm điểm 0–3; nhóm 1 gồm điểm 4–6; nhóm 2 gồm điểm 7–8; nhóm 3 gồm điểm 9–10. Bảng tra `grade_tab` có 11 phần tử kiểu byte, phần tử thứ `s` cho biết nhóm của điểm `s`; mảng đếm `hist` có 4 word, khởi tạo bằng 0.

**Bước 2.3.** Khai báo dữ liệu (mảng điểm cá nhân, `grade_tab`, `hist`) và cài đặt hai thủ tục theo đặc tả giao diện sau. Không có mã khung; tự thiết kế thuật toán.

**Bảng 9.** *Đặc tả giao diện thủ tục*

| Thủ tục | Tham số | Kết quả và ràng buộc |
|---|---|---|
| `stats` | `$a0` = địa chỉ mảng word<br>`$a1` = n (n ≥ 1) | `$v0` = giá trị nhỏ nhất, `$v1` = giá trị lớn nhất. Một lượt duyệt mảng; không ghi bộ nhớ; là thủ tục lá. |
| `histogram` | `$a0` = địa chỉ mảng điểm<br>`$a1` = n<br>`$a2` = địa chỉ `grade_tab`<br>`$a3` = địa chỉ `hist` | Với mỗi điểm `s`, tăng `hist[grade_tab[s]]` thêm 1. Không trả giá trị. Bắt buộc tra bảng bằng `lbu`, không so sánh điểm với ngưỡng. Là thủ tục lá. |

**Bước 2.4.** Viết chương trình chính: nạp tham số, gọi hai thủ tục, in kết quả đúng định dạng sau (hai dòng):

```
min=<giá trị> max=<giá trị>
g0=<số> g1=<số> g2=<số> g3=<số>
```

**Bước 2.5 (DỰ ĐOÁN → QUAN SÁT).** Với mỗi bộ dữ liệu trong Bảng 10, tính tay kết quả trước, sau đó chạy chương trình. Bộ T2 gồm một phần tử {10}; bộ T3 gồm năm phần tử cùng bằng 4.

**Bảng 10.** *Kết quả thống kê*

| Bộ dữ liệu | min / max (tính tay) | g0 g1 g2 g3 (tính tay) | Chương trình in ra |
|---|---|---|---|
| T1: mảng cá nhân (n = 12) | 0 / 10 | 2 5 3 2 | min=0 max=10<br>g0=2 g1=5 g2=3 g3=2 |
| T2: {10} (n = 1) | 10 / 10 | 0 0 0 1 | min=0 max=0<br>g0=0 g1=0 g2=0 g3=1 |
| T3: {4, 4, 4, 4, 4} (n = 5) | 4 / 4 | 0 5 0 0 | min=4 max=4<br>g0=0 g1=5 g2=0 g3=0 |

**Câu 2.3 (Phân tích).** Sau khi `histogram` trả về, người gọi có thể trông cậy vào giá trị của `$a0`–`$a3` hay không? Giải thích bằng quy ước thanh ghi và chỉ ra thanh ghi nào trong bài của bạn đã bị thay đổi. Vì sao hai thủ tục trên không cần lưu `$ra`?

- không trông ậy vào giá trị $a0-$a3 được vì nó là thanh ghi tham số và không được bảo toàn, chúng có thể bị thay đổi.
- thanh ghi bị thay đổi là các thanh ghi $t vì trong bài em dùng các thanh ghi đó để sll, add hoặc lw, còn thanh ghi $a em chỉ dùng để bị đọc, chứ không bị chép hoặc ghi. Em hoàn toán có thể thay đổi giá trị thanh ghi $a mà ko cần chép vào thanh ghi $t.
- trong stats và histogram không có lên jal nên không có $ra, $ra chỉ xuất hiện sau lênh jal trong main, lưu $ra trong stats và histogram là không cần thiết vì nó không cần thêm thủ tục lá.

**Câu 2.4 (Áp dụng).** Đếm số lệnh thực thi trong thân vòng lặp của `histogram` cho một phần tử (kể cả lệnh nhảy). Chỉ ra lệnh nào chuyển chỉ số phần tử thành offset byte đối với mảng word, và vì sao truy cập `grade_tab` không cần lệnh tương tự.

- số lệnh thực thi: 13

- lệnh chuyển chỉ số phần tử thành offset byte là

sll $t1, $t0, 2

Sll $t5, $t4, 2

- truy cập grade_tab không cần lệnh tương tự vì 1 phần tử của nó là 1 byte nên nó không cần tính offset = i * 4 vì 1 word bằng 4 byte nên khi chuyển 1 mảng .word ta cần đổi sang byte nên cần tính offset, bản chất grade_tab là mảng .byte sẵn nên chỉ cần add sau đó lbu là được

**Câu 2.5 (Đánh giá).** Giả sử dữ liệu nhập có một điểm không hợp lệ bằng 11. Phân tích điều gì xảy ra trong `histogram` của bạn (byte được đọc từ đâu, ghi vào đâu). Đề xuất một cách bảo vệ và ước lượng chi phí của nó theo số lệnh thêm vào trên mỗi phần tử.

Với s = 11, histogram đọc grade_tab[11] tại địa chỉ 0x1001005C + 11 = 0x10010067. Địa chỉ này nằm ngoài bảng (bảng chỉ có 11 phần tử, 0x5C–0x66) và là byte đầu của chuỗi str_min, ký tự 'm' = 0x6D = 109. Do đó g = 109, và thủ tục tăng hist[109] tại 0x1001004C + 109×4 = 0x10010200. Địa chỉ này vẫn căn lề word nên MARS không báo lỗi. Kết quả là một ô nhớ ngoài mảng hist bị ghi đè âm thầm, và điểm sai bị mất khỏi thống kê (g0+g1+g2+g3 = n−1).

kiểm tra 0 ≤ s ≤ 10 trước khi tra bảng, nếu không hợp lệ thì bỏ qua phần tử:

```
sltiu $t7, $t2, 11
beq   $t7, $zero, hist_next
```

Dùng sltiu (so sánh không dấu) nên chặn được cả số âm.

chi phí: thêm 2 lệnh mỗi phần tử (13 → 15 lệnh, tăng khoảng 15%).

## TN3 – NGĂN XẾP: KIỂM TRA NGOẶC CÂN BẰNG (60 phút · 3,5 điểm)

### Tình huống

Một thiết bị nhúng (ví dụ bộ điều khiển công nghiệp) cho phép người vận hành nhập công thức hiệu chỉnh cảm biến dưới dạng văn bản, chẳng hạn `{k * (a[i] + b)}`. Trước khi tính toán, phần mềm của thiết bị phải kiểm tra các cặp ngoặc `( )`, `[ ]`, `{ }` có hợp lệ hay không. Nếu thiếu ngoặc hoặc lẫn loại ngoặc mà vẫn tính, công thức bị hiểu sai và thiết bị cho kết quả sai mà không báo lỗi. Trình soạn thảo mã nguồn và trình biên dịch cũng phải làm việc tương tự.

**Bài toán.** Nhận một chuỗi ký tự và trả lời hai câu hỏi: chuỗi có hợp lệ không, và nếu không thì sai ở ký tự thứ mấy, sai theo kiểu nào. Chuỗi hợp lệ khi mỗi ngoặc mở được đóng bởi đúng ngoặc cùng loại và các cặp lồng nhau đúng thứ tự, nghĩa là cặp mở sau phải được đóng trước. Mọi ký tự khác ngoặc đều được bỏ qua.

**Vì sao chỉ đếm số ngoặc là không đủ?** Chuỗi `[(])` có số `[` bằng số `]` và số `(` bằng số `)`, nhưng vẫn sai: ngoặc `(` mở sau cùng lại chưa đóng thì đã gặp `]`. Muốn phát hiện, phải nhớ **thứ tự** các ngoặc mở chưa đóng và luôn đối chiếu ngoặc đóng với ngoặc mở **gần nhất**. Đó chính là tính chất "vào sau, ra trước" của ngăn xếp.

### A. Kiến thức nền: ngăn xếp

**Ngăn xếp (stack)** là cấu trúc dữ liệu hoạt động theo nguyên tắc **LIFO** (last in, first out): phần tử đưa vào sau cùng là phần tử lấy ra đầu tiên, giống chồng đĩa: chỉ đặt thêm hoặc lấy đi ở đỉnh. Hai thao tác cơ bản là **push** (đẩy một phần tử lên đỉnh) và **pop** (lấy phần tử ở đỉnh ra). Ngăn xếp được dùng rất nhiều trong máy tính: lưu dữ liệu tạm khi thủ tục chạy, tính biểu thức, hoàn tác thao tác trong trình soạn thảo, kiểm tra cú pháp. Trong bài này ngăn xếp đóng vai trò cấu trúc dữ liệu của thuật toán.

Trong MIPS, ngăn xếp là một vùng bộ nhớ thông thường được quản lý bằng thanh ghi `$sp` (stack pointer), luôn trỏ tới phần tử ở **đỉnh**. Một số đặc điểm cần nhớ:

- Ngăn xếp **mở rộng về phía địa chỉ thấp**: push làm `$sp` giảm, pop làm `$sp` tăng.
- Mỗi phần tử trong bài này là một word (4 byte), nên `$sp` luôn là bội số của 4.
- Tập lệnh MIPS không có lệnh push hoặc pop riêng; mỗi thao tác gồm **hai lệnh** như bảng dưới. Thứ tự hai lệnh quan trọng: push phải dịch `$sp` trước rồi mới ghi; pop phải đọc trước rồi mới dịch `$sp` lên.

```
# push $t0                  # pop vao $t0
addi $sp, $sp, -4           lw   $t0, 0($sp)
sw   $t0, 0($sp)            addi $sp, $sp, 4
```

Quy tắc cân bằng: tại mọi lối ra của thủ tục, `$sp` phải bằng đúng giá trị lúc thủ tục được gọi. Nếu không, ngăn xếp của người gọi bị lệch và lỗi thường xuất hiện ở nơi cách xa chỗ gây lỗi.

### B. Ví dụ mẫu: push và pop

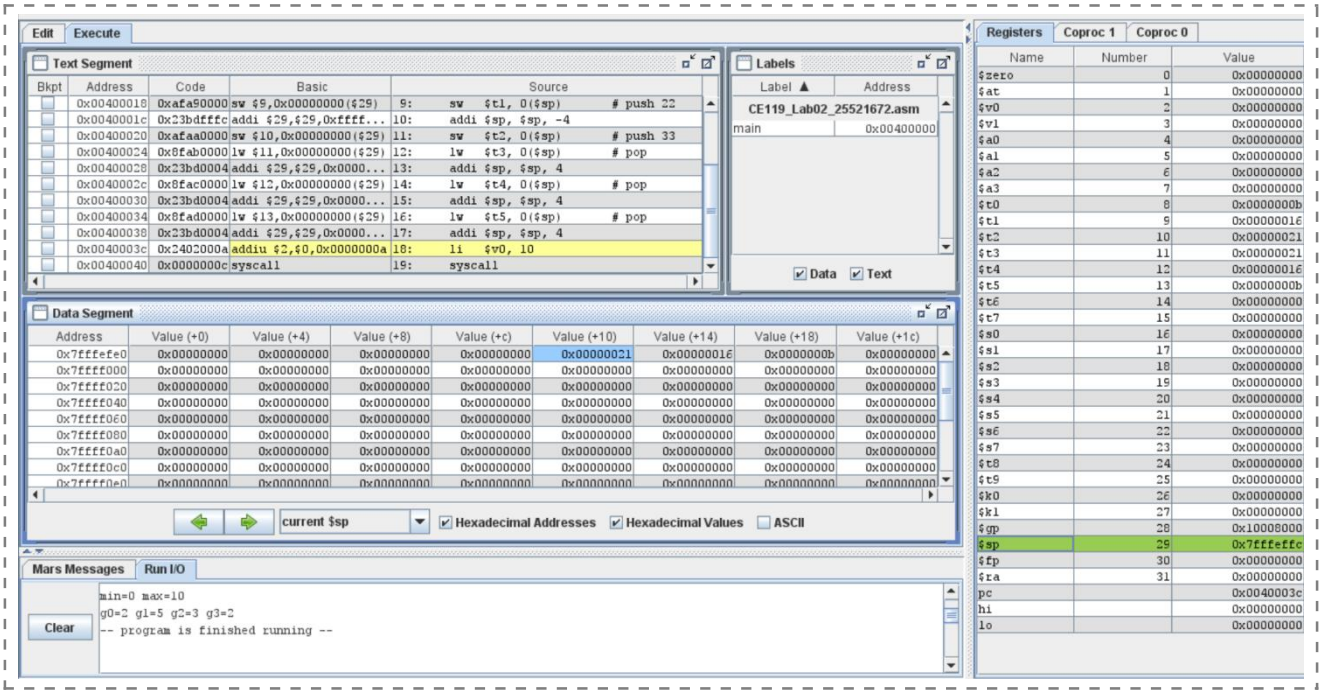
Chương trình dưới đây đẩy ba giá trị 11, 22, 33 lên ngăn xếp rồi lấy ra lần lượt vào `$t3`, `$t4`, `$t5`.

```
.text
main:
    li   $t0, 11
    li   $t1, 22
    li   $t2, 33
    addi $sp, $sp, -4
    sw   $t0, 0($sp)        # push 11
    addi $sp, $sp, -4
    sw   $t1, 0($sp)        # push 22
    addi $sp, $sp, -4
    sw   $t2, 0($sp)        # push 33
    lw   $t3, 0($sp)        # pop
    addi $sp, $sp, 4
    lw   $t4, 0($sp)        # pop
    addi $sp, $sp, 4
    lw   $t5, 0($sp)        # pop
    addi $sp, $sp, 4
    li   $v0, 10
    syscall
```

**Bước 3.1 (DỰ ĐOÁN → QUAN SÁT).** Gọi $S_0$ là giá trị `$sp` lúc bắt đầu (đã ghi ở Bảng 3). Dự đoán `$sp` và nội dung word tại `$sp` sau mỗi thao tác trong Bảng 11, rồi chạy từng bước, chuyển Data Segment sang "current $sp" và ghi giá trị quan sát. Dán ảnh Data Segment sau thao tác pop cuối cùng.

**Bảng 11.** *Trạng thái ngăn xếp sau từng thao tác*

| Sau thao tác | `$sp` dự đoán | `$sp` quan sát | Word tại `$sp` (quan sát) |
|---|---|---|---|
| push 11 | 0x7FFFEFF8 | 0x7FFFEFF8 | 0x0000000B |
| push 22 | 0x7FFFEFF4 | 0x7FFFEFF4 | 0x00000016 |
| push 33 | 0x7FFFEFF0 | 0x7FFFEFF0 | 0x00000021 |
| pop (vào `$t3`) | 0x7FFFEFF4 | 0x7FFFEFF4 | 0x00000016 |
| pop (vào `$t4`) | 0x7FFFEFF8 | 0x7FFFEFF8 | 0x0000000B |
| pop (vào `$t5`) | 0x7FFFEFFC | 0x7FFFEFFC | 0x00000000 |



**Câu 3.1 (Phân tích).** Sau khi pop xong, ba giá trị 11, 22, 33 còn nằm trong bộ nhớ hay đã bị xóa? Giải thích ý nghĩa của thao tác "pop" ở mức lệnh, và nêu một hệ quả đối với việc gỡ lỗi hoặc bảo mật dữ liệu.

– Vẫn còn trong bộ nhớ: sau pop cuối, Data Segment vẫn hiện 33 tại 0x7FFFEFF0, 22 tại 0x7FFFEFF4 và 11 tại 0x7FFFEFF8, trong khi $sp đã về 0x7FFFEFFC.

– Pop gồm lw (đọc word ở đỉnh) và addi $sp, $sp, 4 (dời $sp lên). Không có lệnh nào ghi vào ô nhớ, nên pop chỉ dời đỉnh ngăn xếp chứ không xóa dữ liệu. Ô cũ được coi là trống và chỉ mất khi bị push sau ghi đè.

– Hệ quả: dữ liệu nhạy cảm đã pop vẫn có thể bị đọc lại; muốn xóa thật phải ghi đè (sw $zero, 0($sp)) trước khi pop. Khi gỡ lỗi, nếu lỡ đọc ô dưới $sp thì vẫn ra giá trị trông hợp lệ, nên lỗi khó phát hiện.

## C. Thuật toán kiểm tra ngoặc bằng ngăn xếp

Ý tưởng: duyệt chuỗi từ trái sang phải; gặp ngoặc mở thì push; gặp ngoặc đóng thì pop ngoặc mở ở đỉnh để đối chiếu loại; cuối chuỗi ngăn xếp phải rỗng. Bảng 12 minh họa cách ngăn xếp thay đổi với một chuỗi hợp lệ (đáy ở bên trái, đỉnh ở bên phải).

**Bảng 12.** *Vết thực thi với chuỗi hợp lệ* `[(x)]`

| Vị trí | Ký tự | Hành động | Ngăn xếp sau hành động |
|---|---|---|---|
| 1 | `[` | push `[` | `[` |
| 2 | `(` | push `(` | `[ (` |
| 3 | `x` | bỏ qua | `[ (` |
| 4 | `)` | pop được `(`, cùng loại với `)` | `[` |
| 5 | `]` | pop được `[`, cùng loại với `]` | (rỗng) |
| Hết chuỗi | | ngăn xếp rỗng | hợp lệ |

**Bước 3.2 (DỰ ĐOÁN).** Lập vết thực thi cho chuỗi `{a[b)}` bằng cách điền Bảng 13 theo mẫu Bảng 12. Dừng ở ký tự đầu tiên làm chuỗi sai; ghi kết quả cuối cùng là vị trí lỗi và kiểu lỗi (mô tả bằng lời; mã lỗi được quy định ở phần D).

**Bảng 13.** *Vết thực thi với chuỗi* `{a[b)}`

| Vị trí | Ký tự | Hành động | Ngăn xếp sau hành động |
|---|---|---|---|
| 1 | `{` | push { | { |
| 2 | `a` | Bỏ qua | { |
| 3 | `[` | Push [ | {[ |
| 4 | `b` | Bỏ qua | {[ |
| 5 | `)` | Pop được [, không cùng loại với ) | { |
| Kết luận | | Dừng tại ví trị 5 | Lỗi: ngoặc đóng không cùng loại với ngoặc mở, mã lỗi: 1 -> kết quả (5,1) |

## D. Nhiệm vụ: thủ tục check_brackets

**Bước 3.3.** Cài đặt thủ tục `check_brackets` theo đặc tả sau (không có mã khung; tự thiết kế thuật toán và cách nhận biết ngăn xếp rỗng).

**Bảng 14.** *Đặc tả* `check_brackets`

| Mục | Yêu cầu |
|---|---|
| Tham số | `$a0` = địa chỉ chuỗi `.asciiz`. |
| Kết quả | `$v0` = vị trí (đếm từ 1, tính mọi ký tự) và `$v1` = mã lỗi, theo Bảng 15. |
| Ngăn xếp dữ liệu | Dùng chính ngăn xếp của chương trình (`$sp`); mỗi phần tử chiếm một word (4 byte). Là thủ tục lá. |
| Ràng buộc | Khi return, `$sp` phải bằng giá trị lúc vào thủ tục **trên mọi đường thoát**, kể cả khi phát hiện lỗi giữa chừng lúc ngăn xếp còn phần tử. |

**Bảng 15.** *Mã lỗi và giá trị trả về*

| `$v1` | Ý nghĩa | `$v0` | Ví dụ |
|---|---|---|---|
| 0 | Chuỗi hợp lệ | 0 | `[(x)]` → (0, 0) |
| 1 | Ngoặc đóng không cùng loại với ngoặc mở đang ở đỉnh ngăn xếp | vị trí của ngoặc đóng đó | `(x]` → (3, 1) |
| 2 | Ngoặc đóng khi ngăn xếp rỗng | vị trí của ngoặc đóng đó | `x)` → (2, 2) |
| 3 | Hết chuỗi nhưng ngăn xếp còn ngoặc mở | độ dài chuỗi + 1 | `[x` → (3, 3) |

**Bước 3.4 (DỰ ĐOÁN → QUAN SÁT).** Với mỗi chuỗi trong Bảng 16, tính tay cặp (`$v0`, `$v1`) trước, sau đó chạy. Chuỗi P là chuỗi cá nhân: gồm **k** cặp ngoặc lồng nhau hoàn toàn (k lấy từ Bảng 4), các ngoặc mở lần lượt theo chu kỳ `(`, `[`, `{`, `(`, `[`, `{`, … và các ngoặc đóng đúng loại, đúng thứ tự.

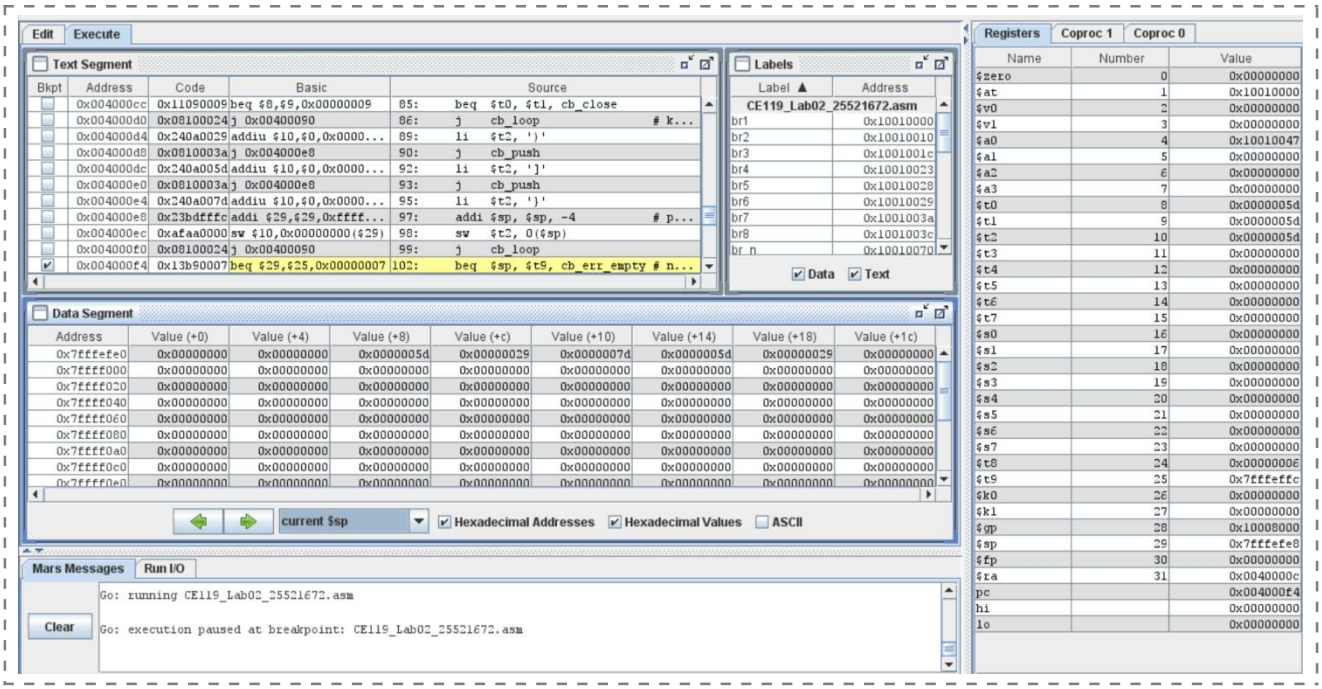
**Bảng 16.** *Kết quả kiểm tra ngoặc*

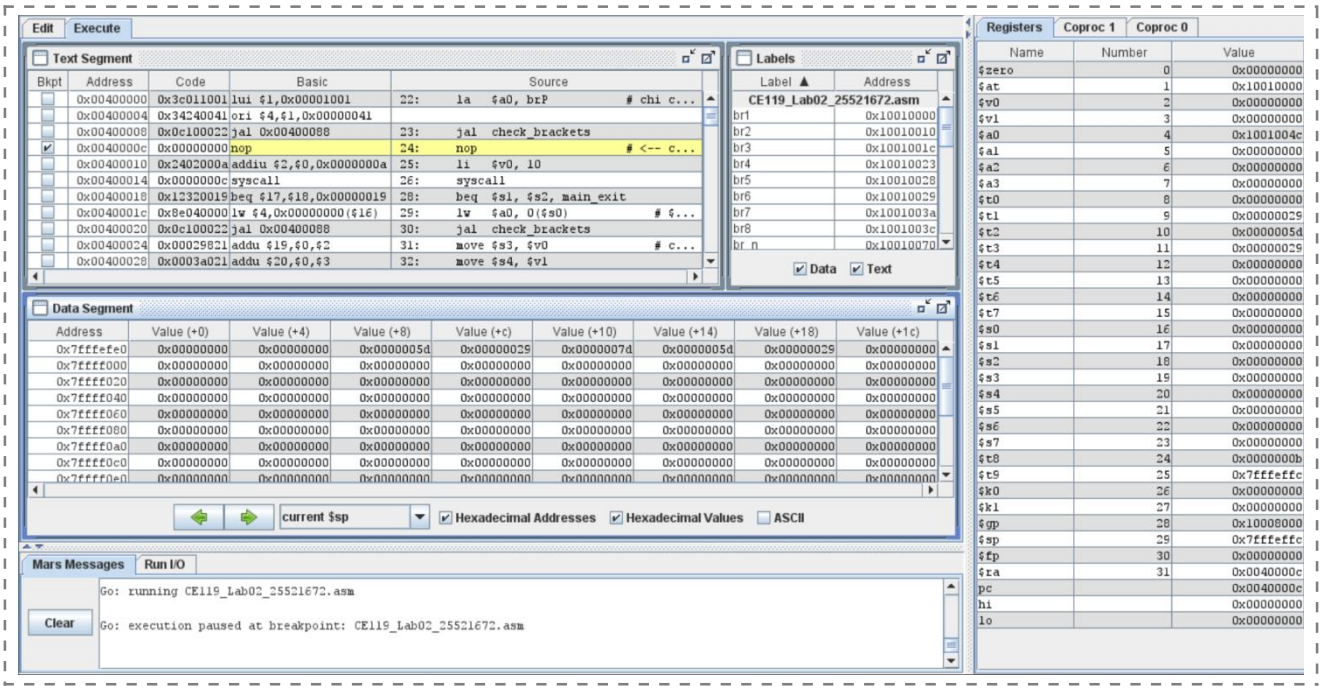
| Bộ | Chuỗi | (`$v0`, `$v1`) tính tay | Chương trình |
|---|---|---|---|
| 1 | `(a+b)*[c-{d/e}]` | 0,0 | 0,0 |
| 2 | `f(x[i]{j}))` | 11,2 | 11,2 |
| 3 | `{[()]}` | 4,1 | 4,1 |
| 4 | `((a)` | 5,3 | 5,3 |
| 5 | `""` (rỗng) | 0,0 | 0,0 |
| 6 | `no brackets here` | 0,0 | 0,0 |
| 7 | `]` | 1,2 | 1,2 |
| 8 | `([)]` | 3,1 | 3,1 |
| P | chuỗi cá nhân | 0,0 | 0,0 |

**Bước 3.5 (DỰ ĐOÁN → QUAN SÁT).** Với chuỗi P, đặt breakpoint tại lệnh đầu tiên xử lý một ngoặc đóng (tức là lúc ngăn xếp sâu nhất). Điền Bảng 17 và dán ảnh Data Segment ở chế độ "current $sp" tại thời điểm đó; sau đó chạy tiếp đến khi thủ tục trả về và chụp lại cùng vùng nhớ.

**Bảng 17.** *Độ sâu ngăn xếp với chuỗi P*

| Đại lượng | Dự đoán | Quan sát |
|---|---|---|
| Số phần tử trên ngăn xếp lúc sâu nhất | 5 | 5 |
| `$sp` lúc sâu nhất (hex) | 0x7FFFEF70 | 0x7FFFEFE8 |
| `$sp` sau khi thủ tục trả về (hex) | 0x7FFFEFFC | 0x7FFFEFFC |



**Câu 3.2 (Đánh giá).** Nêu ít nhất hai cách khôi phục `$sp` trên các đường thoát có lỗi (khi ngăn xếp còn phần tử). So sánh về số lệnh, độ phức tạp và rủi ro sai sót, rồi nêu cách bạn đã chọn và lý do.

Cách 1 – Lưu $sp lúc vào rồi gán lại: đầu thủ tục move $t9, $sp; mọi đường thoát nhảy về nhãn chung cb_exit có move $sp, $t9.
– Số lệnh: cố định, 2 lệnh (1 lúc vào, 1 lúc thoát), không phụ thuộc số phần tử còn trên ngăn xếp.
– Độ phức tạp: thấp, O(1).
– Rủi ro: thấp, chỉ cần đảm bảo mọi nhánh lỗi đều đi qua cb_exit và không dùng $t9 vào việc khác.

Cách 2 – Pop từng phần tử còn lại cho tới khi rỗng: ở mỗi đường thoát lỗi, lặp: nếu $sp ≠ đáy thì addi $sp, $sp, 4.
– Số lệnh: khoảng 3 lệnh × số phần tử còn lại, tức tỉ lệ với độ sâu ngăn xếp.
– Độ phức tạp: O(độ sâu), cần thêm vòng lặp.
– Rủi ro: cao hơn, dễ pop thiếu hoặc thừa, và phải viết hoặc nhảy tới vòng lặp dọn dẹp ở từng nhánh lỗi.

(Cách 3 – Đếm số phần tử đã push trong một biến đếm, khi thoát cộng $sp thêm 4 × đếm: vài lệnh cố định, nhưng phải cập nhật biến đếm ở mỗi push/pop, nên dễ quên.)

Em chọn cách 1: ít lệnh nhất, chi phí không đổi theo độ sâu, và một điểm thoát duy nhất đảm bảo $sp luôn cân bằng. Đồng thời $t9 cũng được dùng để nhận biết ngăn xếp rỗng ($sp == $t9), nên một thanh ghi phục vụ được hai mục đích.

**Câu 3.3 (Phân tích).** Nếu mỗi phần tử chỉ chiếm một byte (dùng `sb` và giảm `$sp` đi 1), chuỗi P tiết kiệm được bao nhiêu byte ngăn xếp? Phương án đó vi phạm đặc điểm nào của `$sp` đã nêu ở phần A, và gây lỗi gì nếu sau đó dùng `lw` hoặc `sw` với địa chỉ dựa trên `$sp`?

– Chuỗi P có độ sâu tối đa 5 phần tử. Dùng word thì cần 5 × 4 = 20 byte, dùng byte thì cần 5 × 1 = 5 byte, nên tiết kiệm được 15 byte.

– Phương án này vi phạm đặc điểm "$sp luôn là bội số của 4" (căn lề word). Ví dụ, sau 1 lần push $sp = 0x7FFFEFFB, và lúc sâu nhất $sp = 0x7FFFEFFC − 5 = 0x7FFFEFF7, đều không chia hết cho 4.

– Hệ quả: nếu sau đó dùng lw hoặc sw với địa chỉ dựa trên $sp (ví dụ lw $t0, 0($sp) hay sw $ra, 0($sp)), địa chỉ không căn lề word, nên MARS báo lỗi runtime "address not aligned on word boundary" và dừng chương trình, giống hiện tượng ở Bước 1.4. Ngoài ra, mọi đoạn mã khác (người gọi, thủ tục khác) vốn giả định $sp căn lề word đều bị ảnh hưởng nếu thủ tục trả về khi $sp còn lệch.

## PHẦN MỞ RỘNG – TRỰC QUAN HÓA PHỔ ĐIỂM (15 phút · cộng 1,0 điểm)

Chỉ thực hiện khi đã hoàn thành các thí nghiệm trên. Viết thủ tục `draw_hist` vẽ phổ điểm dưới dạng biểu đồ cột ngang bằng ký tự `|`: mỗi nhóm xếp loại là một dòng gồm nhãn nhóm, rồi số ký tự `|` bằng số sinh viên của nhóm đó. Ví dụ định dạng (dữ liệu minh họa, không phải kết quả của bạn):

```
g0: |||
g1: |||||
g2: ||||||||
g3: ||||
```

| Thủ tục | Tham số | Yêu cầu |
|---|---|---|
| `draw_hist` | `$a0` = địa chỉ mảng `hist`<br>`$a1` = số nhóm (4) | In đúng định dạng trên cho dữ liệu do `histogram` tính được; nhóm không có sinh viên vẫn in dòng nhãn, không có ký tự `|`. Là thủ tục lá; chương trình chính gọi sau `histogram`. |

Lưu ý: các syscall in ký tự và in số dùng `$v0` và `$a0` để nhận mã và tham số.

Dán ảnh cửa sổ Run I/O với mảng điểm cá nhân (bộ T1):



## KẾT THÚC – NỘP BÀI, PHẢN TƯ VÀ VẤN ĐÁP (20 phút · 2,0 điểm)

### Danh sách kiểm tra trước khi nộp

| ✓ | Mục |
|---|---|
| ✓ | File `CE119_Lab02_<MSSV>.asm` chạy được từ đầu đến cuối, kết thúc bằng syscall thoát. |
| ✓ | Mỗi thủ tục có chú thích đầu (tham số, giá trị trả về, thanh ghi sử dụng) và tuân thủ quy ước thanh ghi. |
| ✓ | Báo cáo `CE119_Lab02_<MSSV>.docx` đã điền đủ các bảng, câu hỏi và ảnh chụp. |
| ✓ | Phần "Liên kết GitHub" và "Tự chấm" ở đầu trang đã điền. |

### Câu hỏi phản tư

**Câu P1 (Đánh giá).** Nêu hai lỗi đáng kể nhất bạn gặp trong buổi thực hành. Với mỗi lỗi: nguyên nhân gốc (gắn với quy ước thanh ghi, bộ nhớ hoặc ngăn xếp), công cụ MARS giúp bạn phát hiện, và cách bạn sửa.

Lỗi 1 – Dùng giá trị làm địa chỉ. Khi in g0–g3, em viết lw $a0, 0($s0), nhưng $s0 đang chứa min = 0 chứ không phải địa chỉ hist (địa chỉ hist được cất ở $s2). Tương tự trong stats, em viết lw $t2, 0($a1), trong khi $a1 là n = 12.
– Nguyên nhân gốc: nhầm giữa địa chỉ và nội dung bộ nhớ. lw cần thanh ghi nền chứa địa chỉ hợp lệ trong vùng .data.
– Phát hiện: MARS dừng chương trình và báo lỗi runtime về địa chỉ không hợp lệ; cửa sổ Registers cho thấy thanh ghi nền chỉ là một số nhỏ (0 hoặc 0xC), không phải dạng 0x1001xxxx.
– Sửa: dùng đúng thanh ghi chứa địa chỉ (lw $a0, 0($s2), lw $t2, 0($a0)). Rút kinh nghiệm: đặt tên và chú thích rõ thanh ghi nào là địa chỉ, thanh ghi nào là giá trị.

Lỗi 2 – Thủ tục phụ thuộc vào thanh ghi do người gọi để lại. Ban đầu stats dùng i do main gán sẵn (li $a2, 0 rồi li $t6, 0 ở main), và dùng addi $t6, $t6, 1 với giả định $t6 = 0.
– Nguyên nhân gốc: vi phạm quy ước thanh ghi. Tham số chỉ được truyền qua $a0–$a3 theo đặc tả, và không được trông cậy vào giá trị của $t. Chương trình vẫn ra đúng vì MARS khởi tạo thanh ghi bằng 0, nhưng lần gọi thứ hai (bộ T2, T3) sẽ sai.
– Phát hiện: chạy Step và theo dõi cửa sổ Registers thấy thủ tục đọc thanh ghi chưa từng được gán bên trong nó; thử gán giá trị rác vào $t trước jal thì kết quả sai.
– Sửa: thủ tục tự khởi tạo mọi biến nội bộ (lw $t1, 0($a0); li $t7, 1) và chỉ dùng $a0, $a1 làm đầu vào.

**Câu P2 (Đánh giá).** Để `check_brackets` báo được vị trí của ngoặc mở chưa đóng (mã 3) thay vì "độ dài chuỗi + 1", cần thay đổi gì ở phần tử đẩy lên ngăn xếp? Ước lượng ảnh hưởng đến bộ nhớ ngăn xếp của chuỗi P.

Hiện tại ngăn xếp chỉ lưu ngoặc đóng mong đợi, nên khi hết chuỗi không biết ngoặc mở nằm ở đâu. Cần lưu thêm **vị trí của ngoặc mở** khi push. Có hai cách:
– Đẩy 2 word cho mỗi ngoặc (ký tự và vị trí): mỗi phần tử 8 byte. Với chuỗi P (sâu 5) cần 5 × 8 = 40 byte thay vì 20 byte, gấp đôi.
– Gói cả hai vào 1 word, ví dụ word = (vị trí << 8) | ký tự: vẫn 4 byte mỗi phần tử, nên P vẫn chỉ dùng 20

byte, đổi lại thêm khoảng 2–3 lệnh để gói và tách (sll/or khi push, andi/srl khi pop).
Khi gặp mã 3, đọc vị trí từ phần tử ở đỉnh (ngoặc mở gần nhất chưa đóng) hoặc ở đáy, tại địa chỉ $t9 − 4 (ngoặc mở đầu tiên chưa đóng), rồi trả về trong $v0.

**Câu P3 (Đánh giá).** Thủ tục `stats` trả về hai giá trị qua `$v0` và `$v1`. Giả sử cần thêm kết quả thứ ba (ví dụ tổng điểm) trong khi quy ước chỉ có hai thanh ghi trả về. Đề xuất ít nhất hai cách và nêu ưu, nhược điểm của từng cách về số lệnh và bộ nhớ.

Cách 1 – Trả qua bộ nhớ: người gọi truyền thêm địa chỉ một biến kết quả trong $a2; stats ghi tổng bằng sw $t, 0($a2).
– Ưu: không giới hạn số kết quả; thủ tục chỉ thêm 1 lệnh sw.
– Nhược: tốn 4 byte bộ nhớ; người gọi thêm la và lw (khoảng 3 lệnh, la là 2 lệnh); thủ tục phải ghi bộ nhớ, trái với ràng buộc "không ghi bộ nhớ" của stats hiện tại.

Cách 2 – Gói hai giá trị nhỏ vào một thanh ghi: điểm thuộc [0, 10] nên min và max đều vừa 1 byte: $v0 = min | (max << 8), còn $v1 = tổng.
– Ưu: không tốn bộ nhớ, vẫn chỉ dùng $v0, $v1.
– Nhược: thêm khoảng 2 lệnh để gói (sll, or) và 2 lệnh để tách (andi, srl) ở người gọi; chỉ dùng được khi giá trị đủ nhỏ, khó đọc và dễ sai.

Cách 3 – Tách thành thủ tục riêng (ví dụ sum_array như ví dụ mẫu 2):
– Ưu: mỗi thủ tục làm một việc, dễ kiểm thử, không tốn bộ nhớ.
– Nhược: phải duyệt mảng thêm một lượt, tốn thêm khoảng 6 lệnh × n phần tử cùng chi phí jal/jr.

**Vấn đáp.** Giảng viên có thể hỏi bất kỳ phần nào của mã và báo cáo đã nộp; sinh viên cần giải thích được lựa chọn thiết kế và tái hiện được kết quả quan sát trên MARS.